

Bomblab 报告

姓名：徐嘉阳

学号：2024201628

总分	Phase_1	Phase_2	Phase_3	Phase_4	Phase_5	Phase_6	secret_phase
70	10	10	10	10	10	10	10

我的scoreboard得分：

2024201628	0	0	0	0	0	0	0
------------	---	---	---	---	---	---	---

具体实现

Phase 1

答案

Step through the gate into Utopia, sink into a world of Melodia.

思路

整体思路是寻址找到目标地址中存储的字符串并输入，即可拆弹。

此阶段调用字符串比较函数，将用户输入的密码与内存中存储的标准字符串作比较，如果一致则拆弹成功。通过汇编代码定位找到内存中存储的正确密码即可：

```
call    1cc7 <strings_not_equal>
lea     0x1d40(%rip),%rsi
```

0x1d40(%rip): RIP 相对寻址，实际地址是当前RIP(0x1440) + 0x1d40 = 0x3180，即正确密码地址。

Phase 2

答案

1031694 913219 1059358 1167140

思路

整体思路是寻址得到两个矩阵的值，计算矩阵乘法结果即可拆弹。

根据汇编的 14d8-14fe 可知此这一段主要实现的功能是**矩阵乘法**，其中最核心的部分是：

```
14db:imul    (%rsi,%rax,8),%edx    # A[i][k] × B[k][j]
14df:add     %edx,%ecx            # 累加结果
```

根据汇编的 1502-1526 可知此阶段的验证规则是逐元素对比输入值与矩阵乘法结果，其中最核心的部分是：

```

1518:mov    0x0(%rbp,%ebx,1),%eax # 取矩阵计算结果
151c:cmp    %eax,(%rsp,%ebx,1)    # 对比计算结果和用户输入
151f:je     150e <phase_2+0xb9>    # 相等则继续验证下一个元素
1521:call   1f2c <explode_bomb>    # 不相等则爆炸

```

下面分析如何确定相乘的两个矩阵 `matA`、`matB`：

```

148e:lea    0x4cab(%rip),%rdi      # 6140 <matA.3> (2x3)
14bb:lea    0x4c5e(%rip),%rsi      # 6120 <matB.2> (3x2)
14d8:mov    (%rdi,%rax,4),%edx      # matA是4字节步长，为int类型
14db:imul   (%rsi,%rax,8),%edx      # matB是8字节步长，分为两个int

```

已知矩阵的起始地址和数据类型，在gdb中通过指定格式输出即可。

Phase 3

答案

7 s 102

思路

整体思路是以 `输入值1` 作为索引进入对应分支，得到对应的 `输入值3` 和最终校验值；根据地址得到掩码的值，掩码值与 `输入值2` 做异或运算的结果与最终校验值相等，以此得到输入值即可拆弹。

根据汇编的 158d-1690 分析此阶段的**分支逻辑**，其中 15a8-1690 是8个分支的具体内容，每个分支的结构都一样，后续将以分支7为例进行分析。首先分析最核心的部分：

```

158d:mov    0x10(%rsp),%eax        # 读取输入值1作为索引
1591:lea    0x1ca8(%rip),%rdx      # 跳转表基地址
1598:movslq (%rdx,%rax,4),%rax      # 计算绝对跳转地址
159c:add    %rdx,%rax           # 计算绝对跳转地址
159f:jmp     %%rax                 # 跳转到对应分支

```

首先分析作为索引的输入值1有什么限制：

```

1582:cmpl    $0x7,0x10(%rsp)      # 检查输入值1是否≤7
1587:ja      1692                  # 大于7则爆炸，即取值为0~7

```

下面令输入值1为7，跳入分支7进行具体分析：

```

167a:mov    $0x73,%eax            # 最终校验值为0x73
167f:cmpl    $0x66,0x14(%rsp)     # 检查输入值3是否等于0x66
1684:je     169c <phase_3+0x158>   # 相等时通过
1686:call   1f2c <explode_bomb>    # 不等时爆炸

```

据此可直接得到，输入值1为7时输入值3一定要为0x66才不会引发爆炸；

下面分析输入值2：

```

1578:mov     0x4b92(%rip),%eax    # 6110 <mask.1>, 用gdb查找可知为32
157e:xor     %al,0xf(%rsp)      # 将输入值2与mask的低8位异或
169c:cmp     %al,0xf(%rsp)      # 比较异或后的输入值2和分支返回的最终校验值
16a0:jne     16b7              # 不相等则爆炸

```

根据异或运算的性质，直接将分支返回的最终校验值与mask进行运算即可得到输入值2，根据ASCII可转化为字符。

Phase 4

答案

31 AC

思路

整体思路是在Phase4中调用func4_1和func4_2，并给这两个函数中的参数赋值，只需要把这些参数带入到两个递归函数计算，得到两个结果作为输入，即可拆弹。

首先分析Phase4：

- 栈的处理和数值的读取等基本操作。
- 数值验证：17b2-17c0 调用func4_1(5)，并将返回值和输入值1进行对比。
- 字符串验证：17d1-1807 调用func4_2(5,7,'A','C','B',buf)，生成字符串并与输入值2进行对比。
- 爆炸分支：181f-183b 提出了几种炸弹爆炸的情况：输入值不是2个；字符串长度不是2；数值或字符串不匹配。

下面分析func4_1和func4_2，这是两个递归函数：

```

int func4_1(int n) {
    if (n <= 0) {
        return 0;
    }
    if (n == 1) {
        return 1;
    }
    // 递归公式: 2 * func4_1(n-1) + 1
    int pre = func4_1(n - 1);
    return 2 * pre + 1;
}

```

```

void func4_2(int n, int s, char c1, char c2, char c3, char* buf) {
    // 递归终止条件: n == 1, 生成2字符字符串
    if (n == 1) {
        buf[0] = c1;
        buf[1] = c2;
        buf[2] = '\0';
        return;
    }

    int pre = func4_1(n - 1); // 调用func4_1计算n-1的结果
    if (pre >= s) {

```

```

        // 分支1: 交换c2和c3递归
        func4_2(n - 1, s, c1, c3, c2, buf);
    } else {
        int next = pre + 1;
        if (next != s) {
            // 分支2: 调整阈值并交换字符递归
            int new_s = s - pre - 1;
            func4_2(n - 1, new_s, c2, c3, c1, buf);
        } else {
            // 分支3: 生成c1c2字符串
            buf[0] = c1;
            buf[1] = c2;
            buf[2] = '\0';
        }
    }
}
}

```

Phase 5

答案

encgh0

思路

整体思路是输入一个长度为6的字符串，对字符串进行遍历，对每个字符取ASCII码，与掩码0xf做与运算（即取**低4位**），再以结果为索引，查array.0数组，将数组值累加最终的和应该是41，反推得到字符串即可拆弹。

分析汇编代码：

首先可得到第一个有效信息，即输入的字符串**长度为6**：

```

1841:mov    %rdi,%rbx          # 记录输入字符串的地址
1844:call   1caa <string_length> # 计算长度
1849:cmp     $0x6,%eax          # 检查字符串长度是否等于6
184c:jne   187a              # 长度不为6时跳转爆炸分支

```

还可得到第二个与数字有关的信息，即**累加结果为41**：

```

1873:cmp     $0x29,%ecx          # 检查累加结果是否等于0x29
1876:jne   1881              # 若不等于跳转爆炸分支

```

下面聚焦与核心的循环逻辑，进行**字符处理和查表求和**：

```

1861:movzbl (%rax),%edx          # 取字符的ASCII码 & 0xf（即取低4位）
1864:and     $0xf,%edx           # 取字符的ASCII码 & 0xf（即取低4位）
1867:add     (%rsi,%rdx,4),%ecx # 把低4位作为索引查表，并将对应数值累加
186a:add     $0x1,%rax         # 指针指向下一个字符
186e:cmp     %rdi,%rax           # 检查是否是字符串末尾
1871:jne   1861              # 若不是末尾，继续循环

```

已知整体思路，下面开始进行反推：
首先利用gdb指令提取 array.0 的完整内容：

索引	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
数值	2	10	6	1	12	16	9	3	4	7	14	5	11	8	15	13

(16+15+4+3+2+1=41,对应索引为5,14,3,7,8,0)
只需从表中选取6个数值，使其和为41，再找到对应的索引，反推ASCII码和对应字符即可。

Phase 6

答案

2 1 5 3 4 6

思路

整体思路是输入6个整数，k表示链表的第k个节点，按照输入顺序重新排序后令链表数值呈现**严格降序**即可拆弹。
首先分析输入格式：

```
18aa:call    1fec <read_six_numbers> # 读取6个整数

18ec:cmp     $0x5,%eax               # 即检查整数≤6
18ef:ja      18ba                      # 若整数>6，爆炸

18ca:mov     (%r12,%rbx,4),%eax       # 存储其他位置的整数
18ce:cmp     %eax,0x0(%rbp)           # 对比当前整数与其他整数是否重复
18d1:jne     18c1                     # 不重复则继续
18d3:call    1f2c <explode_bomb>      # 重复则爆炸
```

综上所述，输入是6个不重复的整数，且每个数都在1~6中，即输入是**1~6的一个排序**。
下面分析汇编进行的操作：
首先分析链表节点查找逻辑：

```
18fc:mov     $0x0,%esi                 # esi循环计数器同时用作索引
1901:mov     (%rsp,%rsi,4),%ecx         # ecx = 第esi个输入整数
1904:mov     $0x1,%eax                  # eax节点计数器
1909:lea     0x4910(%rip),%rdx          # 链表头节点地址
1910:cmp     $0x1,%ecx                  # 检查ecx是不是1
1913:jle     1920                     # 是则直接使用当前节点
1915:mov     0x8(%rdx),%rdx             # 否则取下一个节点
1919:add     $0x1,%eax                # 节点计数器+1
191c:cmp     %ecx,%eax                  # 对比节点计数器与输入整数
191e:jne     1915                     # 不相等则继续查找
1920:mov     %rdx,0x20(%rsp,%rsi,8)     # 将找到的节点地址存入栈
1925:add     $0x1,%esi                # 执行下一个输入的整数
```

这段代码告诉我们一个很重要的信息：**输入整数k时对应定位到链表第k个节点。**
接下来根据 192f-1961 我们可以看出，代码对链表进行了**重组**，按存入栈的顺序进行了连接。换句话说，就是根据输入整数的顺序，将对应的链表节点重新连接成新链表。
随后的验证环节，根据汇编语句可知重组后的链表是**降序排列**：

```
1979:mov    0x8(%rbx),%rax    # rax是当前节点的下一个节点
197d:mov    (%rax),%eax        # eax是下一个节点的值
197f:cmp    %eax,(%rbx)        # 检查当前节点值是否不小于下一个节点值
1981:jge    1970              # 满足降序则继续
1983:call    1f2c <explode_bomb> # 不满足则爆炸
```

最后我们要做的就是用gdb获取链表节点的值，按照对应序号的大小顺序进行重排即可得到输入。
(注意我们已知node1的地址，利用gdb可以获取链表当前节点值和下一节点地址，以此类推)

node1	node2	node3	node4	node5	node6
842	885	321	272	600	246

secret_phase

答案

```
33113
```

思路

整体思路是先找到secret_phase的入口，根据func7的汇编代码还原移动规则，得到移动方式的数字索引并修正坐标，找到从起点移动的终点的组合即可拆弹。

触发secret_phase入口：分析phase_defused 函数的汇编代码。
触发条件：通过前面6个普通Phase，并在输入中包含一个特定的字符串。

```
2175:cmpl    $0x6,0x45bc(%rip)    # 检查是否通过了6个phase
217c:je      2183 <phase_defused+0x1c> # 通过6个phase则继续
...
21f1:call    1cc7 <strings_not_equal> # 字符串比较
21f6:test    %eax,%eax
21f8:jne    21c2 <phase_defused+0x5b> # 不相等则退出
2217:call    1bd3 <secret_phase>    # 相等调用secret_phase
```

触发关键词：hidden。
根据phase_defused 汇编代码的逻辑，我们可以找到关键词的存储地址，用gdb指令读取即可。

```
21ea:lea     0x1450(%rip),%rsi      # 3641 <array.0+0x3e1>
```

分析地图：分析func7函数的汇编代码。
地图大小：8x8，代码中有明确提示，如大小比较，同时适配数据表结构。

```
1b14:cmp    $0x7,%esi      # 检查x坐标是否≤7
1b17:jbe    1b58 <func7+0x1ac>
```

地图结构：用gdb指令读取汇编代码中的row0-row7可以知道地图中障碍的位置（1）。

Y\X	0	1	2	3	4	5	6	7
0	0	0	1	0	0	1	0	0
1	0	0	0	1	0	0	0	1
2	1	0	1	0	0	1	0	0
3	1	0	0	0	0	0	0	0
4	0	1	0	0	1	0	1	0
5	1	0	0	1	1	0	0	0
6	0	0	0	0	0	1	0	1
7	0	1	0	0	0	0	0	0

分析 19cb-1ac2 的数据表结构，利用gdb指令分别读取前面的X、Y坐标偏移表和后面的X、Y坐标修正表。
结合 X/Y 偏移表，每个索引对应**坐标偏移**：

索引	X偏移	Y偏移	实际方向
0	-2	1	左上斜向
1	-1	2	左上大斜向
2	1	2	右上大斜向
3	2	1	右上斜向
4	2	-1	右下斜向
5	1	-2	右下大斜向
6	-1	-2	左下大斜向
7	-2	-1	左下斜向

根据输入的索引移动后，如果目标位置是障碍（1），要根据索引进行**坐标修正**，X/Y修正表如下：

索引	0	1	2	3	4	5	6	7
X修正量	-1	0	0	1	1	0	0	-1
Y修正量	0	1	1	0	0	-1	-1	0

分析输入：获取上述基本信息后对输入进行分析。
输入格式：根据 secret_phase 函数可知输入是长度不超过20的字符串。

```
1beb:call    string_length    # 计算字符串长度
1bf0:cmp     $0x14,%eax       # 长度>20时爆炸
```

输入内容：这一阶段拆弹的关键是递归调用func7函数，下面将核心部分翻译为c语言代码。

```
int func7(const char *str, int param2, int param3, int pos) {
    // 拆弹条件：递归传参满足param2==4 且 param3==7
    if (param2 == 4 && param3 == 7) {
        return 1;
    }
    // 合法性校验（长度超限或字符串结束还未满足条件就爆炸）
    if (pos > 19 || str[pos] == '\0') {
        return 0;
    }
    // 计算索引k（ASCII码低3位，此处我们只选0~7作为输入，输入本身就是对应的索引）
    int k = (unsigned char)str[pos] & 0x7;
    // 执行偏移表，得到临时坐标
    int temp_param2 = param2 + table1[k];
    int temp_param3 = param3 + table2[k];
    // 计算障碍表索引
    int row_idx = (temp_param2) & 0x7;
    int col_idx = (temp_param3) & 0x7;
    // 检查是否遇到障碍
    int new_param2, new_param3;
    if (row[row_idx][col_idx] == 1) {
        // 遇到障碍，执行修正表
        new_param2 = temp_param2 + table3[k];
        new_param3 = temp_param3 + table4[k];
    } else {
        // 没有遇到障碍则不修正
        new_param2 = temp_param2;
        new_param3 = temp_param3;
    }
    // 不能越界
    if ((new_param2 | new_param3) > 7) {
        return 0;
    }
    // 递归调用
    return func7(str, new_param2, new_param3, pos + 1);
}
```

根据这段核心逻辑，我们可以知道：

- 需要输入一串长度不超过20的字符串，每一个字符串都在0~7之间。
- 输入的每一个字符都将作为索引递归调用表格中数据，更新坐标。
- 初始的坐标为（0,0），递归调用后最终坐标为（4,7）。
- 更新坐标的规则是先调用X/Y偏移表，更新临时坐标，若对应位置障碍为1，再调用X/Y修正表更新坐标，否则无需调用X/Y修正表。
- 过程中坐标不能越界，即要一直在0~7范围内。

根据简单的计算和试验即可得到答案。

反思总结

- secret_phase明显比其他几个题难，花费了很多的时间，中间一直没搞清楚后16位常数表的使用规则，所幸最后解出来了。
- 在防爆的时候一开始觉得打断点麻烦，有可能粗心忘记导致爆炸，所以直接把函数改了。结果输入错误答案不仅不会爆炸还直接通过了（惊），最后老老实实打断点了。
- 对汇编指令和gdb都有了进一步的了解。

参考资料

课本《深入理解计算机系统》

柴老师的ppt